





আন্তর্জাতিক ইসলামী বিশ্ববিদ্যালয় চট্টগ্রাম  
الجامعة الإسلامية العالمية شيتاغونغ  
International Islamic University Chittagong

## Department of Computer Science and Engineering

### PROJECT PROPOSAL

Project Title:

### Flutter GetX Portfolio Application

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| Student Name:   | Md Imtiaz Uddin Chowdhury                                        |
| ID:             | C233170                                                          |
| Course Title:   | Software Development II Lab                                      |
| Course Code:    | CSE-3644                                                         |
| Course Teacher: | Shahriar Rahman Inan<br>Adjunct Lecturer,<br>Dept. Of CSE (IIUC) |

Approval of the Course Teacher

---

Shahriar Rahman Inan  
Adjunct Lecturer,  
Dept. Of CSE (IIUC)

## 1. Abstract

This report documents the design, development, and implementation of a cross-platform personal portfolio application built using the Flutter framework together with the GetX state management library, and developed in the FlutLab online Flutter IDE. The application presents the developer's profile, skills, a short biography, and a curated list of projects, along with a fully functional contact section that links directly to email, phone, and map applications. It demonstrates practical use of reactive (.obs) state management, a clean single-responsibility widget structure, external URL launching, and a custom tap-triggered size-and-colour transition animation on the profile picture. The application is built from a single Dart codebase and is configured to run across Android, iOS, and Web targets.

## 2. Introduction

### 2.1 Objective

The objective of this lab assignment was to design and implement a clean, production-style Flutter application that presents a personal developer portfolio, while applying core Software Development concepts covered in the SD2 course - namely reactive state management, modular widget decomposition, and interaction with device-level services (email, phone dialer, and maps) through the platform's URL handling.

### 2.2 Motivation

A personal portfolio application was chosen because it is a compact but complete showcase of real mobile-app concerns: presenting structured static data (skills, projects, contact details) through dynamic widgets, wiring up real external actions (mailto, tel, and map links) instead of static placeholder text, keeping the UI responsive across mobile and web viewport widths, and adding a small custom animation to make the interface feel alive - all of which map directly to concepts covered in the SD2 lab.

### 2.3 Scope

- 1) Present the developer's profile photo, name, title, and a short bio.
- 2) Display the developer's technical skill set as a styled chip list.
- 3) Showcase a list of projects, each opening its live/deployed link in an external browser.
- 4) Provide a working 'Let's Connect' section with tappable Email, Phone, and Location entries.
- 5) Animate the profile avatar's size and border colour on tap.
- 6) Gracefully fall back to a placeholder icon if the profile image fails to load.
- 7) Keep the layout readable on both narrow mobile screens and wider web/desktop viewports.

### 3. Tools & Technologies Used

| Category         | Technology / Package                                            |
|------------------|-----------------------------------------------------------------|
| Language         | Dart (SDK >=3.8.0 <4.0.0)                                       |
| Framework        | Flutter (Material 3)                                            |
| State Management | GetX (get: ^4.7.2)                                              |
| External Links   | url_launcher: ^6.3.2 (email, phone, Google Maps, project links) |
| Iconography      | cupertino_icons: ^1.0.8 + built-in Material Icons               |
| Linting          | flutter_lints: ^5.0.0                                           |
| IDE              | FlutLab (online Flutter IDE, flutlab.io)                        |
| Target Platforms | Android, iOS, Web                                               |

### 4. System Architecture

The application follows a lightweight GetX-based layering: Models (data), Controllers (state and business logic), Views (screen composition), and Widgets (reusable UI pieces). Because the app is a single-screen portfolio, no named-route navigation or dependency-injection bindings were required; HomeController is instantiated directly via Get.put() inside the view, keeping the structure simple while still separating UI from logic.

#### 4.1 Folder Structure

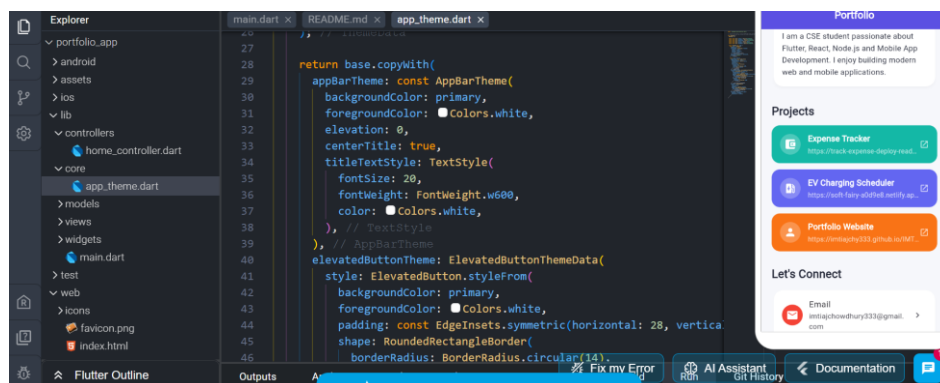


Fig 1: lib/ folder structure viewed in the FlutLab explorer

**lib/** main.dart — Application entry point

**core/** — App-wide theme configuration (AppTheme)

**models/** — Plain Dart data model (Project)

**controllers/** — GetxController exposing reactive state and static content (HomeController)

**views/** — The single portfolio screen (HomePage)

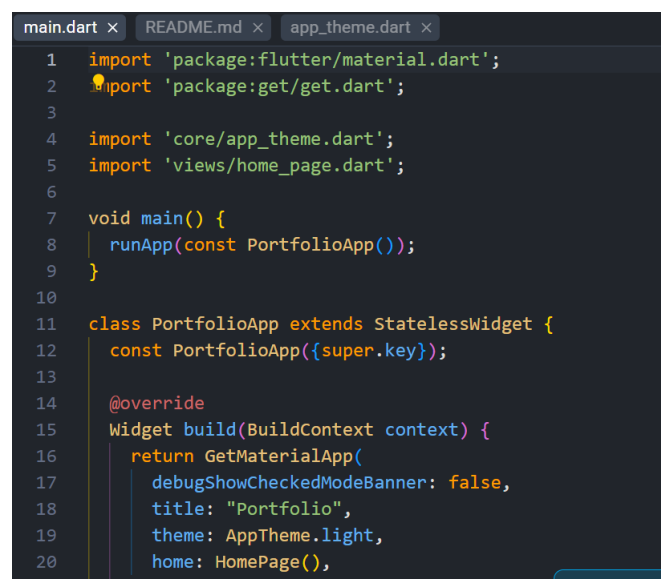
**widgets/** — Reusable UI components (ProfileAvatar, SkillsCard, SectionTitle, ProjectCard, ContactSection)

## 4.2 Layer Responsibilities

| Layer       | Responsibility                                                                                                                                                                                                                     |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Models      | Project - a plain Dart class holding a project's title, URL, icon, and accent colour.                                                                                                                                              |
| Controllers | HomeController extends GetxController; exposes reactive (.obs) flags for the avatar animation and image-load failure, static profile/contact/skill/project data, and action methods (openUrl, sendEmail, callPhone, openLocation). |
| Views       | HomePage composes the screen from widgets, reading controller state via Obx() for the animated avatar.                                                                                                                             |
| Widgets     | Stateless, single-purpose components: ProfileAvatar, SkillsCard, SectionTitle, ProjectCard, and ContactSection.                                                                                                                    |

## 4.3 Application Bootstrap Flow

main.dart initializes the app by running PortfolioApp, a GetMaterialApp configured with the light theme from AppTheme and HomePage as its home screen. HomePage instantiates HomeController with Get.put() and lays out the profile avatar, name and title, a 'Contact Me' button, the skills card, the About Me section, the list of projects, and the 'Let's Connect' section inside a width-constrained, scrollable column so the same code renders sensibly on both phone and web widths.



```
main.dart x README.md x app_theme.dart x
1 import 'package:flutter/material.dart';
2 import 'package:get/get.dart';
3
4 import 'core/app_theme.dart';
5 import 'views/home_page.dart';
6
7 void main() {
8   runApp(const PortfolioApp());
9 }
10
11 class PortfolioApp extends StatelessWidget {
12   const PortfolioApp({super.key});
13
14   @override
15   Widget build(BuildContext context) {
16     return GetMaterialApp(
17       debugShowCheckedModeBanner: false,
18       title: "Portfolio",
19       theme: AppTheme.light,
20       home: HomePage(),
21     );
22   }
23 }
```

Fig 2: main.dart - application entry point

## 5. Key Features

### 5.1 Animated Profile Avatar

Tapping the profile picture toggles a reactive flag in HomeController. ProfileAvatar listens to this flag inside an Obx() and drives an AnimatedContainer that grows the avatar from 140 to 180 logical pixels while its border and glow shadow transition from the primary to the secondary theme colour over 700 ms with an easeOutBack curve - a combined size-and-colour transition triggered purely by user interaction.

## 5.2 Resilient Image Loading

The avatar image is loaded with `Image.asset` and a custom `errorBuilder`. If the asset fails to load, the controller's `avatarFailed` flag is set (deferred to the next frame via `addPostFrameCallback`) and the UI falls back to a tinted circular icon instead of a blank or broken image.

## 5.3 Real Contact Actions

The 'Let's Connect' section is not static text - each tile calls `url_launcher` to open the device's mail client (`mailto:`), phone dialer (`tel:`), or Google Maps (with the location string URL-encoded into a search query), with a `snackbar` shown if no matching app is available or the launch fails.

## 5.4 Dynamic Project List

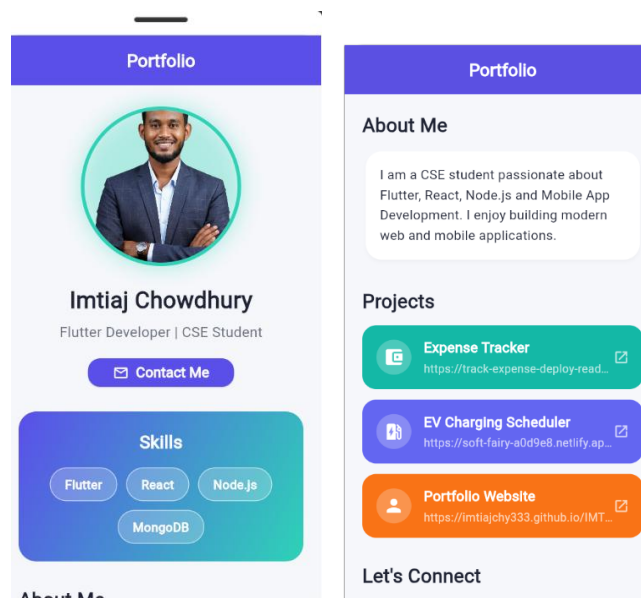
Projects are modelled as data (`Project`: title, url, icon, colour) inside `HomeController` and rendered through a single `ProjectCard` widget via `.map()`, so adding a new project is a one-line change rather than a new hand-built widget. Tapping a card opens its deployed URL in an external browser.

## 5.5 Responsive Layout

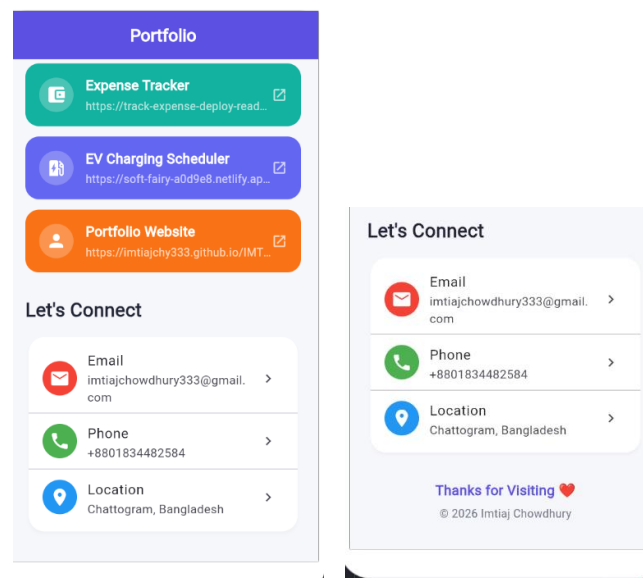
The whole page is wrapped in a `ConstrainedBox` with a `maxWidth` of 560 logical pixels inside a centered, scrollable column, so the layout stays readable and card-like on wide web/desktop viewports instead of stretching edge-to-edge, while behaving like a normal single-column layout on mobile.

## 6. Application Screens

### 1) Home Section (Profile & Skills) and About Me / Projects Section



## 2) Projects & Let's Connect (scrolled view) and Let's Connect Section



## 7. Testing

Manual functional testing was performed on the FlutLab live preview for:

- 1) Each project card opening its correct deployed URL in an external browser.
- 2) The Email, Phone, and Location tiles correctly launching mailto, tel, and Google Maps intents respectively.
- 3) The profile avatar animating its size and border colour on every tap, in both directions.
- 4) The layout remaining readable and centered at both narrow (mobile) and wide (web) preview widths.
- 5) The fallback avatar icon appearing correctly when the profile image asset was deliberately renamed.

## 8. Challenges Faced

- 1) Synchronising the avatar's size and colour changes on a single AnimatedContainer so both properties transition smoothly together rather than one lagging behind the other.
- 2) Handling a failed profile-image load without triggering a setState-during-build error, solved by deferring the state update with addPostFrameCallback.
- 3) Keeping the single-column layout usable across drastically different widths (mobile vs. web) using a centered ConstrainedBox instead of a full responsive grid.
- 4) Wiring three different URL schemes (mailto, tel, and a Maps search query) through one shared launch helper with consistent error feedback via Get.snackbar.

## 9. Reflection on SD2 Lab Learning

| SD2 Lab Concept       | Implementation in This Project                                                                                                     |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------|
| State Management      | Applied GetX (GetXController, .obs, Obx) to drive the avatar animation reactively without setState().                              |
| Custom Widgets        | Decomposed the UI into single-purpose reusable widgets - ProfileAvatar, SkillsCard, SectionTitle, ProjectCard, and ContactSection. |
| Animations            | Implemented a custom tap-triggered size-and-colour transition using AnimatedContainer and a reactive boolean flag.                 |
| Data Modelling        | Represented portfolio projects as a plain Dart model (Project) rendered generically instead of hardcoded widgets.                  |
| Platform Integration  | Used url_launcher to trigger real email, phone, and maps actions from the UI.                                                      |
| Responsive UI Design  | Used a centered, width-constrained scroll layout so the same code works on mobile and web.                                         |
| Error Handling        | Added an image error fallback and snackbar feedback for failed external-link launches.                                             |
| Software Architecture | Organised the project into Models, Controllers, Views, and Widgets layers following good separation of concerns.                   |

## 10. Conclusion

This project reflects the practical knowledge gained throughout the SD2 laboratory course. It applies GetX for reactive state management, a custom tap-triggered animation, real platform integration through url\_launcher, and a clean, modular widget architecture. Building the portfolio application in FlutLab reinforced how a small, well-structured Flutter codebase can present dynamic content, respond to user interaction, and connect to external services across Android, iOS, and Web from a single project.

## 11. References

- 1) Flutter Documentation. <https://docs.flutter.dev>
- 2) GetX Package Documentation. <https://pub.dev/packages/get>
- 3) URL Launcher Package. [https://pub.dev/packages/url\\_launcher](https://pub.dev/packages/url_launcher)
- 4) FlutLab Online Flutter IDE. <https://flutlab.io>
- 5) Material Design 3 Guidelines. <https://m3.material.io>